

Python E-Course

Module 6 — Image Enhancement

Detailed Notes

Module Overview

Level: Enhancement

Duration: ~1 week

Prerequisites: Modules 1-5

What You'll Learn

- Read and interpret image histograms.
- Apply histogram equalization (global) and CLAHE (adaptive).
- Tune brightness and contrast cleanly.
- Apply gamma correction for perceptual fixes.
- Use LUTs (lookup tables) for arbitrary tone mapping.

Lessons

#	Lesson	Duration
1	Histograms — Reading and Plotting	30 min
2	Histogram Equalization	30 min
3	CLAHE (Adaptive Equalization)	30 min
4	Brightness & Contrast	25 min
5	Gamma Correction	25 min
6	LUTs — Lookup Tables	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 7: Smoothing & Filtering

Lesson 1: Histograms — Reading and Plotting

Module: 6 — Image Enhancement

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Compute an image histogram with `cv2.calcHist` or NumPy.
- Plot histograms (grayscale and per-channel color).
- Identify dark / bright / low-contrast / clipped images from their histograms.

Prerequisites

- Modules 1-5, matplotlib basics.

1. Why This Matters

The histogram of an image is its "fingerprint." A glance tells you whether it's too dark, too bright, has low contrast, or has clipped highlights. Every enhancement decision is informed by reading a histogram first.

2. Concept

2.1 What a histogram counts

For each possible intensity (0..255 for uint8), how many pixels have that value. A grayscale histogram is a length-256 array.

2.2 Compute with NumPy

```
hist, edges = np.histogram(img, bins=256, range=(0, 256))
# hist.shape = (256,)
```

2.3 Compute with OpenCV

```
hist = cv2.calcHist([img], channels=[0], mask=None, histSize=[256], ranges=[0, 256])
# hist.shape = (256, 1)
```

`cv2.calcHist` is faster on large images and supports multi-channel directly.

2.4 Plot it

```
import matplotlib.pyplot as plt
plt.plot(hist)
plt.xlim(0, 256); plt.xlabel("intensity"); plt.ylabel("count")
plt.show()
```

2.5 Per-channel for color

```
for i, color in enumerate(["b", "g", "r"]):
    h = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(h, color=color)
```

2.6 What you're looking for

- Concentrated on the left → image is dark.
- Concentrated on the right → image is bright.
- Narrow range (e.g. 80-180) → low contrast.
- Spike at 0 or 255 → clipping (lost detail in shadows / highlights).
- Bimodal (two peaks) → likely foreground + background — good for thresholding.

2.7 Cumulative distribution function (CDF)

Sum of the histogram. Used by histogram equalization (Lesson 2). For a perfectly uniform distribution, the CDF is a straight diagonal line.

```
cdf = hist.cumsum()
cdf_normalized = cdf * float(hist.max()) / cdf.max()    # for overlaying on the
histogram plot
```

3. Code Examples

Example 1: Grayscale histogram

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import coins

img = coins()
hist = cv2.calcHist([img], [0], None, [256], [0, 256])

plt.plot(hist); plt.xlim(0, 256)
plt.title("coins() histogram"); plt.xlabel("intensity"); plt.ylabel("count")
plt.show()
print("min", img.min(), "max", img.max(), "mean", img.mean())
```

Expected Output: A histogram peaked roughly in the 30-90 intensity range (dark background) with a secondary mode around 130-180 (the coins). Console prints range and mean confirming this.

Example 2: RGB / BGR per-channel histogram

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)

for i, color in enumerate(["b", "g", "r"]):
    h = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(h, color=color, label=color)
plt.xlim(0, 256); plt.legend(); plt.title("coffee per-channel"); plt.show()
```

Expected Output: Three overlaid curves; the red channel is shifted toward higher intensities (lots of warm coffee tones), while blue is concentrated lower.

Example 3: Identify a dark image

```
import cv2, numpy as np

# Dim copy of a bright image
img = (cv2.cvtColor(__import__("skimage.data", fromlist=["camera"]).camera(),
                    cv2.COLOR_GRAY2BGR) * 0.3).astype(np.uint8)

print("mean:", img.mean())
# Most pixels in [0..80] -> heavy left skew, "dark image" diagnosis
```

Expected Output: mean: ~35 and a histogram bunched on the left — diagnostic for "needs brightening".

Example 4: CDF overlay

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import coins

img = coins()
hist = cv2.calcHist([img], [0], None, [256], [0, 256]).ravel()
cdf = hist.cumsum()
cdf_n = cdf * hist.max() / cdf.max()

plt.plot(hist, color='gray'); plt.plot(cdf_n, color='red', label="CDF")
plt.xlim(0, 256); plt.legend(); plt.title("coins histogram + CDF"); plt.show()
```

Expected Output: The gray histogram with a red CDF rising from 0 to the histogram's peak; you can read the percentage of pixels below any intensity by looking at the red curve.

4. Parameter Sensitivity

Param	Effect
bins	More bins = finer resolution but spikier
range	Restrict to a sub-band (e.g. only mid-tones)

5. Common Mistakes

Mistake	What Happens	Fix
Wrap arg as img not [img] in cv2.calcHist	TypeError	Wrap in list
Computing histogram of color image as one channel	Wrong / misleading	Per-channel loop or convert to gray
Forgetting xlim(0, 256)	Plot has lots of empty space	Set the limit

6. Practice Tasks

- Plot the grayscale histogram of coins().
- Plot per-channel histograms for coffee().
- Make a copy of camera() darkened by $\times 0.3$. Plot the histogram. Diagnose "dark".

7. Key Takeaways

- Histogram = pixel-value distribution.

- Shape tells you about brightness, contrast, clipping, bimodality.
- `cv2.calcHist([img], [0], None, [256], [0, 256])` is the standard call.
- CDF is the sum — used by equalization.

8. What's Next

Histogram equalization — automatic global contrast boost driven by the CDF.

Lesson 2: Histogram Equalization

Module: 6 — Image Enhancement

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Apply `cv2.equalizeHist` on grayscale.
- Understand the CDF-based mapping behind it.
- Recognise when global equalization fails (uneven lighting → CLAHE next).

Prerequisites

- Module 6 Lesson 1

1. Why This Matters

A one-line contrast booster. Often the cheapest way to make a dim image readable. But "global" — it can wash out bright regions while fixing dark ones.

2. Concept

2.1 The idea

Map intensities so the histogram becomes as uniform as possible. The mapping function = the normalised CDF.

If your image has 10% of pixels below intensity 50, equalization maps intensity 50 to intensity 25 (10% of 255 $\approx$ 25.5). After the map, the new histogram is roughly flat.

2.2 The math

$$\text{mapping}(v) = \text{round}((\text{CDF}(v) - \text{CDF}_{\min}) / (N - \text{CDF}_{\min}) \times 255)$$

Where N is the total pixel count. OpenCV does this for you.

2.3 The function

```
eq = cv2.equalizeHist(gray)
```

Input must be (H, W) `uint8`.

2.4 Color images — don't equalize per channel

`cv2.equalizeHist` on each B/G/R channel separately produces color shifts. Better:

```
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
ycc[..., 0] = cv2.equalizeHist(ycc[..., 0])
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
```

Equalize only the Y (luma) channel — fixes contrast without color drift.

2.5 Pure-NumPy equivalent

```
def equalize(gray):
    hist, _ = np.histogram(gray, 256, (0, 256))
    cdf = hist.cumsum()
    cdf_m = np.ma.masked_equal(cdf, 0)
    cdf_m = (cdf_m - cdf_m.min()) * 255 / (cdf_m.max() - cdf_m.min())
    cdf_final = np.ma.filled(cdf_m, 0).astype('uint8')
    return cdf_final[gray]
```

2.6 Where it fails

If different regions of the image have very different lighting (bright window, dark interior), global equalization brightens shadows but blows out highlights. CLAHE (Lesson 3) fixes this.

3. Code Examples

Example 1: Basic equalize

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import moon

img = moon()
eq = cv2.equalizeHist(img)

fig, ax = plt.subplots(2, 2, figsize=(10, 8))
ax[0,0].imshow(img, cmap='gray'); ax[0,0].set_title("original")
ax[0,1].imshow(eq, cmap='gray'); ax[0,1].set_title("equalized")
ax[1,0].hist(img.ravel(), 256, [0, 256]); ax[1,0].set_title("orig histogram")
ax[1,1].hist(eq.ravel(), 256, [0, 256]); ax[1,1].set_title("eq histogram")
for a in ax.ravel(): a.set_xlim(0, 256) if "hist" not in a.get_title() else None
plt.tight_layout(); plt.show()
```

Expected Output: Top-left: original moon (mid-gray, low contrast). Top-right: same shape with much more visible craters / shadows. Bottom: histograms — original is narrow / centred; equalized is spread across 0..255.

Example 2: Color equalize via YCrCb

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
ycc[..., 0] = cv2.equalizeHist(ycc[..., 0])
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
cv2.imwrite("coffee_eq.png", out)
```

Expected Output: Coffee image with boosted contrast — shadows darker, highlights brighter, colors approximately preserved (not as washed out as per-channel BGR equalization would be).

Example 3: Per-channel BGR is wrong

```
import cv2, numpy as np
from skimage.data import coffee
```



```
img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
wrong = cv2.merge([cv2.equalizeHist(c) for c in cv2.split(img)])
cv2.imwrite("coffee_eq_wrong.png", wrong)
```

Expected Output: Coffee image with oversaturated, slightly-off colors — proof that per-channel equalization shifts hue.

Example 4: Pure-NumPy version

```
import numpy as np
from skimage.data import moon

def equalize(gray):
    hist, _ = np.histogram(gray, 256, (0, 256))
    cdf = hist.cumsum()
    cdf_m = np.ma.masked_equal(cdf, 0)
    cdf_m = (cdf_m - cdf_m.min()) * 255 / (cdf_m.max() - cdf_m.min())
    return np.ma.filled(cdf_m, 0).astype('uint8')[gray]

eq = equalize(moon())
print(eq.min(), eq.max(), eq.mean())
```

Expected Output: 0 255 ~127 — eq spans full range, mean near the midpoint of the histogram.

4. Parameter Sensitivity

Equalization itself has no params — but YCrCb-based color equalization preserves color much better than naive per-channel.

5. Common Mistakes

Mistake	What Happens	Fix
Equalize each BGR channel	Color shift / oversaturation	Equalize Y of YCrCb, or L of LAB
Apply on already-uniform image	No visible change	Save the cycles
Use on uneven-lit images	Highlights blown out	Use CLAHE (next lesson)
Forget input must be uint8	TypeError	Cast first

6. Practice Tasks

- Equalize moon(); show before/after histograms.
- Equalize the Y channel of coffee() via YCrCb.
- Implement the pure-NumPy equivalent and confirm output matches cv2.equalizeHist.

7. Key Takeaways

- cv2.equalizeHist flattens grayscale histograms based on the CDF.
- Apply on luma (Y) for color images — never per BGR channel.
- Global → great for uniform images, bad for uneven lighting.

8. What's Next

CLAHE — adaptive equalization that handles uneven lighting.

Lesson 3: CLAHE (Adaptive Equalization)

Module: 6 — Image Enhancement

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.createCLAHE` for adaptive contrast enhancement.
- Tune `clipLimit` and `tileGridSize`.
- Decide when CLAHE beats global `equalizeHist`.

Prerequisites

- Module 6 Lesson 2

1. Why This Matters

CLAHE = Contrast Limited Adaptive Histogram Equalization. It equalizes per-tile so different regions can have different contrast adjustments. Result: a window scene with a bright sky and a dark interior gets a fix that works for both.

2. Concept

2.1 How it works (intuition)

- Split image into small tiles (e.g. 8×8 grid).
- Equalize each tile's histogram independently.
- Bilinearly interpolate between neighbours to avoid visible tile boundaries.
- Clip the histogram peaks before equalization so noise doesn't get amplified.

2.2 OpenCV API

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
out = clahe.apply(gray)
```

2.3 Params

Param	Effect
<code>clipLimit</code>	How aggressively to clip histogram peaks. Higher = more contrast, more noise. Typical 1.0–4.0.
<code>tileGridSize</code>	Tiles per axis. Smaller (e.g. 8×8) = more local. Larger (e.g. 16×16) = closer to global.

2.4 Color images — Y channel only

Same recipe as plain `equalize`:

```
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
```

```
ycc[..., 0] = clahe.apply(ycc[..., 0])
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
```

Or do CLAHE on L of LAB — same idea, slightly different perceptual flavour.

2.5 Why it beats global

Global equalize uses one mapping for the whole image. CLAHE uses a different (but smoothly interpolated) mapping per region — handles a bright window and a dark wall in the same image.

3. Code Examples

Example 1: Global vs CLAHE on uneven lighting

```
import cv2
import matplotlib.pyplot as plt
from skimage.data import moon

img = moon()
eq = cv2.equalizeHist(img)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8)).apply(img)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(img, cmap='gray'); ax[0].set_title("original")
ax[1].imshow(eq, cmap='gray'); ax[1].set_title("equalizeHist")
ax[2].imshow(clahe, cmap='gray'); ax[2].set_title("CLAHE")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Three moons. Original is low contrast. equalizeHist over-amplifies the bright crater regions. CLAHE produces a balanced result with subtle detail across the surface.

Example 2: Color CLAHE

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(lab)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
L_eq = clahe.apply(L)
out = cv2.cvtColor(cv2.merge([L_eq, a, b]), cv2.COLOR_LAB2BGR)
cv2.imwrite("coffee_clahe.png", out)
```

Expected Output: Coffee photo with localised contrast boost — shadows in the cup's interior become more visible without over-blowing the cup's rim highlights.

Example 3: Tune clipLimit

```
import cv2
from skimage.data import moon

img = moon()
for cl in [1.0, 2.0, 4.0, 8.0]:
```

```
clahe = cv2.createCLAHE(clipLimit=cl, tileGridSize=(8, 8))
cv2.imwrite(f"moon_clahe_cl{int(cl)}.png", clahe.apply(img))
```

Expected Output: Four versions. cl=1.0 is subtle; cl=8.0 is aggressive with visible noise in flat areas.

Example 4: Tune tile size

```
import cv2
from skimage.data import moon

img = moon()
for ts in [(4, 4), (8, 8), (16, 16)]:
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=ts)
    cv2.imwrite(f"moon_clahe_t{ts[0]}.png", clahe.apply(img))
```

Expected Output: 4×4 shows visible blocky transitions in flat areas; 16×16 is closer to global; 8×8 is the usual sweet spot.

4. Parameter Sensitivity

Param	Low	High
clipLimit	subtle, no noise	aggressive, can amplify noise
tileGridSize width	very local, blocky	smoother, closer to global

5. Common Mistakes

Mistake	What Happens	Fix
Apply CLAHE per BGR channel	Color shift	Use L of LAB or Y of YCrCb
clipLimit too high	Noise amplified	Try 2.0-3.0 first
tileGridSize too small	Visible tile artefacts	Stick with (8, 8) usually
Input is not uint8	Error	Convert first

6. Practice Tasks

- Apply CLAHE to moon(); compare with global equalizeHist.
- Try clipLimit ∈ {1, 3, 6}; pick which looks best.
- Apply CLAHE to the L channel of coffee() (LAB).

7. Key Takeaways

- cv2.createCLAHE(clipLimit, tileGridSize).apply(gray).
- Defaults: clipLimit 2–4, tile 8×8.
- For color: do CLAHE on Y (YCrCb) or L (LAB), not per BGR.
- Handles uneven lighting that global equalize can't.

8. What's Next

Brightness and contrast — the explicit knobs, often combined with CLAHE.

Lesson 4: Brightness & Contrast

Module: 6 — Image Enhancement

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Adjust brightness and contrast safely with `cv2.convertScaleAbs`.
- Build an interactive trackbar version.
- Understand why "scale and shift" is the simplest contrast model.

Prerequisites

- Module 6 Lesson 3

1. Why This Matters

Brightness/contrast knobs are sometimes all an image needs. The math is one line; the trick is understanding what you're adjusting and clipping correctly.

2. Concept

2.1 Linear adjustment

```
out = clip( $\alpha$  * img +  $\beta$ , 0, 255)
```

- α (alpha) = contrast scale. >1 = more contrast. <1 = less.
- β (beta) = brightness offset (added intensity).

2.2 OpenCV's helper

```
out = cv2.convertScaleAbs(img, alpha=1.5, beta=20)
```

It does the math, clips to 0–255, and casts to `uint8` in one call.

2.3 Why "scale and shift" is the contrast model

Scaling by α stretches the histogram around 0; shifting by β slides it. To centre contrast around mid-gray (128), use:

```
out = clip( $\alpha$  * (img - 128) + 128 +  $\beta$ , 0, 255)
```

(This is essentially what photo apps' "contrast" slider does.)

2.4 Interactive trackbars

```
cv2.createTrackbar("alpha x10", "win", 10, 30, lambda v: None)
cv2.createTrackbar("beta", "win", 0, 100, lambda v: None)
```

Then read with `cv2.getTrackbarPos`.

3. Code Examples

Example 1: Constant brightness boost

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
out = cv2.convertScaleAbs(img, alpha=1.0, beta=40)
cv2.imwrite("brighter.png", out)
```

Expected Output: Coffee image, +40 on all pixels (saturated at 255 where needed) — looks brighter overall.

Example 2: Contrast scale

```
import cv2
from skimage.data import camera

img = cv2.cvtColor(camera(), cv2.COLOR_GRAY2BGR)
out = cv2.convertScaleAbs(img, alpha=1.6, beta=0)
cv2.imwrite("contrast.png", out)
```

Expected Output: Cameraman with darker shadows and brighter highlights — overall punchier.

Example 3: Contrast around mid-gray

```
import cv2, numpy as np
from skimage.data import camera

img = camera().astype(np.float32)
alpha, beta = 1.6, 0
out = alpha * (img - 128) + 128 + beta
out = np.clip(out, 0, 255).astype(np.uint8)
cv2.imwrite("contrast_centered.png", out)
```

Expected Output: Similar to Example 2 but contrast is symmetric around 128 — useful when you want to preserve overall brightness.

Example 4: Trackbar tuner

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
cv2.namedWindow("tune")
cv2.createTrackbar("alpha x10", "tune", 10, 30, lambda v: None)
cv2.createTrackbar("beta", "tune", 0, 100, lambda v: None)

while True:
    a = cv2.getTrackbarPos("alpha x10", "tune") / 10
    b = cv2.getTrackbarPos("beta", "tune")
    out = cv2.convertScaleAbs(img, alpha=a, beta=b)
    cv2.imshow("tune", out)
    if cv2.waitKey(30) & 0xFF == ord('q'):
```

```
        break
    cv2.destroyAllWindows()
```

Expected Interaction: A window with two sliders; the image updates live as you drag.

4. Parameter Sensitivity

Param	Lower	Higher
α	flatter / less contrast	punchier; clipping risk
β	darker	brighter; clipping risk

5. Common Mistakes

Mistake	What Happens	Fix
<code>img * 1.5</code> on uint8	float result; not displayable directly	Use <code>cv2.convertScaleAbs</code>
Not clipping after manual math	Bright pixels wrap to 0	Always <code>np.clip(0, 255)</code> before cast
Treating α as additive	Wrong knob	α multiplies, β adds

6. Practice Tasks

- Apply $\alpha=1.5$, $\beta=20$ to `coffee()`. Save.
- Apply $\alpha=0.7$, $\beta=-30$ (less contrast, dim). Save.
- Build the trackbar example and find a setting you like.

7. Key Takeaways

- `out =  $\alpha$  * img +  $\beta$` , then clip + cast.
- `cv2.convertScaleAbs(img, alpha, beta)` handles all of it.
- Centring contrast around 128 preserves average brightness.

8. What's Next

Gamma correction — non-linear brightness adjustment matching human vision.

Lesson 5: Gamma Correction

Module: 6 — Image Enhancement

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Apply gamma correction.
- Choose $\gamma < 1$ (brighten shadows) vs $\gamma > 1$ (darken / boost highlights).
- Implement gamma using a precomputed LUT for speed.

Prerequisites

- Module 6 Lesson 4

1. Why This Matters

Linear brightness adjustment is rarely what your eye wants. Human vision is non-linear — we discriminate darker tones much better than lighter ones. Gamma correction matches this behaviour. It's also what your monitor does internally to compensate for sRGB encoding.

2. Concept

2.1 The formula

```
out = ((in / 255) **  $\gamma$ ) * 255
```

Per pixel. After clipping and casting to uint8.

- $\gamma = 1.0 \rightarrow$ identity (no change)
- $\gamma < 1.0 \rightarrow$ brighten dark areas (shadows lifted)
- $\gamma > 1.0 \rightarrow$ darken dark areas (more contrast in highlights)

2.2 Visual intuition

- A dark image (mean ~ 50)? Use $\gamma \approx 0.4$ – 0.6 .
- A washed-out image (mean ~ 190)? Use $\gamma \approx 1.5$ – 2.2 .

2.3 Implement with a LUT (fast)

For uint8 there are only 256 possible input intensities — compute their mapped outputs once, then index:

```
import numpy as np, cv2

def gamma_correct(img, gamma):
    lut = np.array([(i / 255.0) ** gamma) * 255 for i in range(256)],
                  dtype=np.uint8)
    return cv2.LUT(img, lut)
```

cv2.LUT applies the lookup table to every pixel — much faster than $(img / 255) ** \gamma$ for large images.

2.4 Why sRGB matters

Most images on disk are stored with an implicit gamma (~ 2.2). When you do arithmetic in pixel space (blending, averaging), you're working with the encoded values — not perceived linear light. For perceptually-correct blending, convert to linear (divide by 255, raise to 2.2), do math, convert back. For most apps, the encoded space is fine.

3. Code Examples

Example 1: Brighten shadows

```
import cv2, numpy as np
from skimage.data import camera

def gamma_correct(img, g):
    lut = np.array([(i / 255.0) ** g) * 255 for i in range(256)], dtype=np.uint8)
    return cv2.LUT(img, lut)

img = camera()
for g in [0.4, 0.7, 1.0, 1.5, 2.2]:
    out = gamma_correct(img, g)
    cv2.imwrite(f"camera_g{g}.png", out)
```

Expected Output: Five copies. $g=0.4$ is very bright (shadows lifted heavily). $g=1.0$ is unchanged. $g=2.2$ is very dark.

Example 2: Color gamma

```
import cv2, numpy as np
from skimage.data import coffee

def gamma_correct(img, g):
    lut = np.array([(i / 255.0) ** g) * 255 for i in range(256)], dtype=np.uint8)
    return cv2.LUT(img, lut)

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
cv2.imwrite("coffee_g0_5.png", gamma_correct(img, 0.5))
cv2.imwrite("coffee_g2_0.png", gamma_correct(img, 2.0))
```

Expected Output: Two versions — $g=0.5$ looks washed-out/bright; $g=2.0$ looks dark and moody.

Example 3: Compare gamma vs linear brightness

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
linear = cv2.convertScaleAbs(img, alpha=1.0, beta=50)
gamma = np.array([(i/255.0)**0.5)*255 for i in range(256)], dtype=np.uint8)[img]

print("means: linear", linear.mean(), " gamma", gamma.mean())
```

Expected Output: Both brighten the image, but linear adds 50 everywhere (clipping highlights), while gamma lifts shadows more than mid-tones (no clipping at the top). Means differ slightly.

4. Parameter Sensitivity

γ	Effect
0.3-0.6	strong lift of dark regions
0.7-0.9	mild lift
1.0	no change
1.1-1.5	mild darken of dark regions
1.5-2.5	strong darken; punchier highlights

5. Common Mistakes

Mistake	What Happens	Fix
Forgetting to divide by 255 first	Massive overflow	The formula is $(i/255)^\gamma * 255$
Using gamma on a normalised float	Already correct space — double-correction	Pick one space and stick with it
Doing pixel-by-pixel python loop	Slow	Build a LUT once

6. Practice Tasks

- Apply $\gamma=0.5$ to `camera()`; save and inspect.
- Apply $\gamma=2.0$; save.
- Implement `gamma_correct` once, then time it on a 4000×3000 image — should be fast thanks to LUT.

7. Key Takeaways

- Gamma: $\text{out} = (\text{in}/255)^\gamma * 255$.
- $\gamma < 1$ brightens shadows; $\gamma > 1$ darkens / boosts highlights.
- Use a LUT for speed.

8. What's Next

LUTs in general — any tone mapping you can imagine.

Lesson 6: LUTs — Lookup Tables

Module: 6 — Image Enhancement

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Build a lookup table for any per-pixel function.
- Apply LUTs with `cv2.LUT`.
- Implement common effects: negative, posterize, sepia, threshold-like maps.

Prerequisites

- Module 6 Lesson 5

1. Why This Matters

Any per-pixel function with `uint8` inputs can be precomputed into a 256-entry table. `cv2.LUT` then applies it across the whole image in microseconds, no matter how complex the function.

2. Concept

2.1 The LUT

```
lut = np.array([f(i) for i in range(256)], dtype=np.uint8)
out = cv2.LUT(img, lut)
```

- `f(i)` defines what each input intensity maps to.
- `lut.dtype` must match `img.dtype` (typically `uint8`).
- For color: pass a (256, 3) LUT for per-channel maps, or a single (256,) applied to each channel.

2.2 Useful presets

Negative:

```
lut = np.arange(255, -1, -1, dtype=np.uint8)
```

Posterize (4 levels):

```
n = 4
step = 256 // n
lut = np.array([min((i // step) * step, 255) for i in range(256)], dtype=np.uint8)
```

Sepia (per-channel LUTs for BGR):

```
def sepia(img):
    # Convert to grayscale and tint
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Tint: B 50, G 100, R 150 mapping (approximate sepia)
    tint = np.zeros_like(img)
```

```

tint[..., 0] = (gray * 0.40).astype(np.uint8)
tint[..., 1] = (gray * 0.65).astype(np.uint8)
tint[..., 2] = (gray * 0.85).astype(np.uint8)
return tint

```

(For sepia you usually combine LUT with a small linear transform — LUT alone can do it via per-channel LUTs.)

2.3 Why faster than direct math

cv2.LUT is $O(N)$ but with one cheap memory access per pixel. A Python loop over pixels is hundreds of times slower; even vectorised `f(img)` may compute repeated values for the same input intensity. LUT precomputes 256 outputs, then looks them up.

2.4 Per-channel LUTs (color)

```

b_lut = ... # (256,) uint8
g_lut = ...
r_lut = ...
img[..., 0] = cv2.LUT(img[..., 0], b_lut)
img[..., 1] = cv2.LUT(img[..., 1], g_lut)
img[..., 2] = cv2.LUT(img[..., 2], r_lut)

```

Or build a (256, 3) LUT and pass it as one.

3. Code Examples

Example 1: Negative

```

import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
lut = np.arange(255, -1, -1, dtype=np.uint8)
neg = cv2.LUT(img, lut)
cv2.imwrite("astro_negative.png", neg)

```

Expected Output: Photographic negative of astronaut — bright becomes dark and vice versa.

Example 2: Posterize

```

import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
n = 4
step = 256 // n
lut = np.array([min((i // step) * step, 255) for i in range(256)], dtype=np.uint8)
post = cv2.LUT(img, lut)
cv2.imwrite("coffee_poster.png", post)
print("unique values per channel after posterize:", len(np.unique(post[..., 0])))

```

Expected Output: Coffee image with banded "posterized" colour regions (only 4 levels per channel). Console prints 4 for unique values.

Example 3: Step-threshold via LUT

```
import cv2, numpy as np
from skimage.data import coins

img = coins()
lut = np.where(np.arange(256) > 128, 255, 0).astype(np.uint8)
binary = cv2.LUT(img, lut)
cv2.imwrite("coins_binary.png", binary)
```

Expected Output: Coins image converted to pure black/white (intensity above 128 → 255; below → 0). Note: same as `cv2.threshold(..., 128, 255, cv2.THRESH_BINARY)`.

Example 4: Sepia-ish tone via per-channel LUT

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
b_lut = np.clip(np.arange(256) * 0.40, 0, 255).astype(np.uint8)
g_lut = np.clip(np.arange(256) * 0.65, 0, 255).astype(np.uint8)
r_lut = np.clip(np.arange(256) * 0.85, 0, 255).astype(np.uint8)

out = img.copy()
out[..., 0] = cv2.LUT(out[..., 0], b_lut)
out[..., 1] = cv2.LUT(out[..., 1], g_lut)
out[..., 2] = cv2.LUT(out[..., 2], r_lut)
cv2.imwrite("astro_sepia.png", out)
```

Expected Output: Astronaut tinted with warm brown / sepia tones — like an old photograph.

4. Parameter Sensitivity

(Any LUT is a 256-entry array — sensitivity = the function you choose.)

5. Common Mistakes

Mistake	What Happens	Fix
LUT not uint8	Wrong output dtype	<code>dtype=np.uint8</code>
LUT not length 256	Error	Build exactly 256 entries
Applying to non-uint8 image	Behavior undefined	Convert input to uint8 first
Building LUT each frame in a loop	Slow	Build once, reuse

6. Practice Tasks

- Build a posterize LUT for 8 levels. Apply to `coffee()`.
- Build a "boost contrast" LUT: stretch range 50..200 to 0..255. Apply to `camera()`.
- Build the sepia 3-channel LUT effect.

7. Key Takeaways

- LUT = 256-entry table mapping input → output for each intensity.
- `cv2.LUT(img, lut)` is the fastest way to apply any per-pixel function.

- Use for negatives, posterize, thresholds, gamma, sepia, custom tone curves.

8. What's Next

End of Module 6. Next: Module 7 — Smoothing & Filtering — the heart of image processing.

Module 6 — Exercises

18 exercises.

6.1 Histograms

- (E) Plot grayscale histogram of `coins()`.
- (M) Plot per-channel histogram of `coffee()`.
- (H) Darken `camera()` $\times 0.3$ and inspect histogram — diagnose "dark".

6.2 Histogram Equalization

- (E) Equalize `moon()`; show before/after.
- (M) Equalize Y of YCrCb on `coffee()`.
- (H) Implement pure-NumPy equalize; compare to OpenCV.

6.3 CLAHE

- (E) Apply CLAHE to `moon()`. Compare with `equalizeHist`.
- (M) Try `clipLimit` $\in \{1, 3, 6\}$.
- (H) Apply CLAHE to L of LAB on `coffee()`.

6.4 Brightness/Contrast

- (E) `cv2.convertScaleAbs(coffee, alpha=1.5, beta=20)`.
- (M) Center contrast around 128 (manual NumPy).
- (H) Build the trackbar tuner.

6.5 Gamma

- (E) $\gamma=0.5$ on `camera()`.
- (M) $\gamma=2.0$ on `coffee()`.
- (H) Implement `gamma_correct` with LUT; measure timing on a big image.

6.6 LUTs

- (E) Photographic negative LUT on `astronaut()`.
- (M) 8-level posterize LUT on `coffee()`.
- (H) 3-channel sepia LUT.

Module 6 — Quiz

10 MCQs.

Q1

A histogram peaked near 0 indicates: A. bright image B. dark image C. high contrast D. low contrast

Answer: B (L1)

Q2

The CDF is: A. cumulative sum of histogram B. derivative C. log D. inverse

Answer: A (L1, L2)

Q3

cv2.equalizeHist works on: A. color images directly B. grayscale uint8 C. float D. binary

Answer: B (L2)

Q4

For color equalization, apply to: A. each BGR channel B. only B C. Y of YCrCb (or L of LAB) D. RGB after conversion

Answer: C (L2)

Q5

CLAHE is better than global equalize when: A. image is dark B. lighting is uneven C. image is small D. image is grayscale

Answer: B (L3)

Q6

clipLimit in CLAHE controls: A. tile size B. how aggressively to limit histogram peaks C. clipping range D. color space

Answer: B (L3)

Q7

cv2.convertScaleAbs(img, alpha=1.5, beta=20): A. $(1.5 \cdot \text{img} + 20)$ clipped to 0..255 as uint8 B. multiplies hue C. converts color space D. blurs

Answer: A (L4)

Q8

For gamma $\gamma < 1$, the result is: A. darker B. brighter (shadows lifted) C. inverted D. unchanged

Answer: B (L5)

Q9

LUTs are useful because: A. they save memory B. they're vectorised per-pixel mapping C. they handle floats D. they convert formats

Answer: B (L6)

Q10

A "negative" can be done with a LUT containing: A. `np.arange(256)` B. `np.arange(255, -1, -1)` C. `np.zeros(256)` D. `np.ones(256) * 128`

Answer: B (L6)

Module 6 — Assignment: Auto-Enhance Tool

Estimated time: 2 hours

Combines: Module 6 + Modules 4-5

Brief

Build an auto-enhance CLI that picks brightness / contrast / gamma adjustments based on the image's histogram statistics.

Requirements

- `enhance.py INPUT [--output OUT] [--mode auto|clahe|gamma]`
- auto mode:
- Compute mean and std.
- If $\text{mean} < 80 \rightarrow$ apply CLAHE on luma (YCrCb Y).
- If $\text{mean} > 180 \rightarrow$ apply $\gamma = 1.4$.
- If $80 \leq \text{mean} \leq 180$ and $\text{std} < 40 \rightarrow$ light contrast boost ($\alpha=1.3$, $\beta=0$).
- Else $\rightarrow$ no-op.
- clahe mode: always apply CLAHE on Y (`clipLimit=3`, `tile=(8,8)`).
- gamma mode: apply γ based on auto rule.
- Print: chosen mode + reasoning + before/after mean/std.
- Save the result.

Constraints

- Module 6 + prior concepts only.
- Use functions named per-step.
- Argparse.

Expected Output

```
$ python enhance.py too_dark.jpg
mean before: 38.4   std before: 22.1
chosen mode: clahe (mean below 80)
mean after : 124.7  std after : 56.3
saved -> too_dark_enhanced.jpg
```

Grading Rubric

Criterion	Weight
Auto-mode logic	30%
CLAHE path	20%
Gamma path	20%
Reporting	15%
Style	15%

Submission

Save as `assignment_module6.py`.

Module 6 — Mini Project: Histogram-Driven Interactive Editor

Overview

A small cv2-window editor with trackbars for brightness, contrast, gamma, and CLAHE. Real-time preview alongside the histogram of the current output.

Concepts Used

- All of Module 6 (histograms, CLAHE, brightness/contrast, gamma).
- Trackbars (M2 L3).

Features

- 5 trackbars: brightness β ($-100..100$), contrast $\alpha \times 10$ ($5..30$), gamma $\times 10$ ($3..30$), CLAHE on/off (0/1), CLAHE clipLimit $\times 10$ ($5..80$).
- Window shows the result + an inline matplotlib-style histogram strip below.
- s saves; q quits.

Starter Code

hist_editor.py:

```
import sys, cv2, numpy as np

def histogram_strip(gray, width):
    hist = cv2.calcHist([gray], [0], None, [256], [0, 256]).ravel()
    hist = (hist / hist.max() * 80).astype(np.int32)
    strip = np.full((100, width, 3), 30, dtype=np.uint8)
    for x in range(256):
        col = int(x * width / 256)
        cv2.line(strip, (col, 100), (col, 100 - int(hist[x])), (200, 200, 200), 1)
    return strip

def apply_pipeline(img, beta, alpha, gamma, clahe_on, clahe_clip):
    out = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)
    lut = np.array([((i/255)*gamma)*255 for i in range(256)], dtype=np.uint8)
    out = cv2.LUT(out, lut)
    if clahe_on:
        ycc = cv2.cvtColor(out, cv2.COLOR_BGR2YCrCb)
        clahe = cv2.createCLAHE(clipLimit=clahe_clip, tileGridSize=(8, 8))
        ycc[..., 0] = clahe.apply(ycc[..., 0])
        out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
    return out

def run(path):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)

    cv2.namedWindow("editor")
    cv2.createTrackbar("brightness+100", "editor", 100, 200, lambda v: None)
    cv2.createTrackbar("contrast x10", "editor", 10, 30, lambda v: None)
```

```

cv2.createTrackbar("gamma x10",      "editor", 10, 30, lambda v: None)
cv2.createTrackbar("clahe (0/1)",    "editor", 0, 1, lambda v: None)
cv2.createTrackbar("clip x10",       "editor", 30, 80, lambda v: None)

while True:
    b = cv2.getTrackbarPos("brightness+100", "editor") - 100
    a = cv2.getTrackbarPos("contrast x10",    "editor") / 10
    g = cv2.getTrackbarPos("gamma x10",       "editor") / 10
    c = cv2.getTrackbarPos("clahe (0/1)",     "editor")
    cl = cv2.getTrackbarPos("clip x10",        "editor") / 10
    out = apply_pipeline(img, b, a, g, c == 1, cl)
    gray = cv2.cvtColor(out, cv2.COLOR_BGR2GRAY)
    strip = histogram_strip(gray, out.shape[1])
    combo = np.vstack([out, strip])
    cv2.imshow("editor", combo)
    k = cv2.waitKey(30) & 0xFF
    if k == ord('q'):
        break
    if k == ord('s'):
        cv2.imwrite("edited.png", out); print("saved -> edited.png")
cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")

```

Expected Interaction

```

$ python hist_editor.py coffee.png
(window opens with the image on top and a histogram strip below)
move sliders; histogram updates in real time
press s -> saved -> edited.png
press q -> quit

```

Extension Challenges

- Add a "before/after" toggle (b).
- Add per-channel histograms in the strip.
- Add undo / redo.

Grading Rubric

Criterion	Weight
All trackbars work	25%
Pipeline correct	25%
Histogram strip works	20%
Save / quit	10%
Style	20%